COL1000: Introduction to Programming

Lecture 20

Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi

Time Complexity

What is a good algorithm?

Should be correct on all inputs

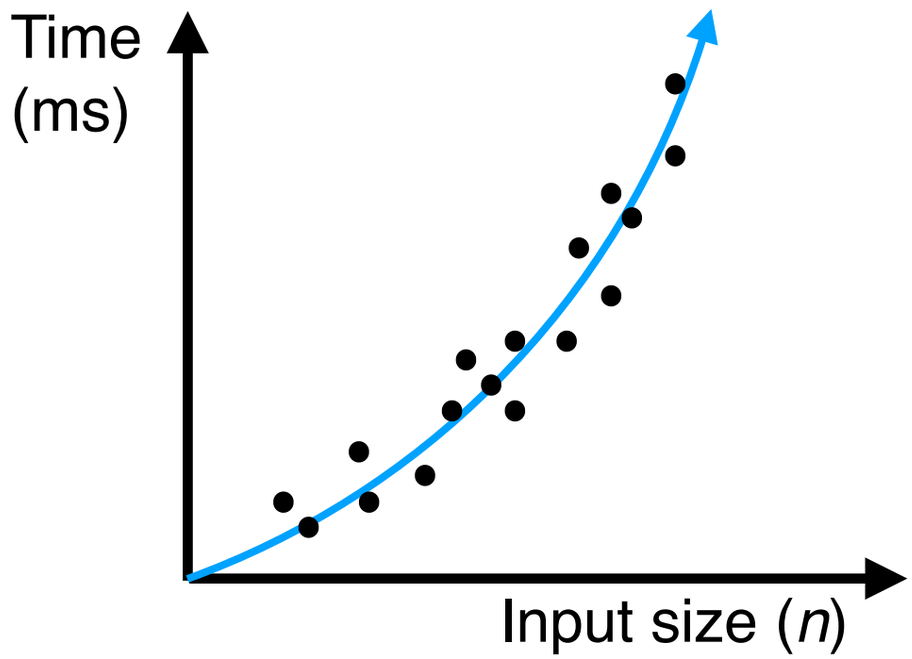
- **Efficient:**
  - Fast running time
  - Take less space

- **Efficiency will be function of input size.**

  Example: Number of bits, Number of data points

How should we measure running time?

- **Experimental study:**
  - Write a program that implements the algorithm.
  - Run the program with data sets of varying size and composition.
  - Use a system call to get a measure of the actual running time.



Limitations of Experimental Study

- It is necessary to implement and test the algorithm in order to determine its running time.

- Experiments can be performed only on a limited set of inputs.
  May not be indicative of the running time on other inputs not included in the experiment.

- In order to compare two algorithms, the same hardware and software environments are needed.

Beyond Experimental Studies

We take an alternate methodology for analyzing running time of algorithms:

- By inspecting the code, we count the **number of primitive/basic** operations executed by the code.

- **Primitive Operations:** These are low-level **atomic operations** like
  - assignment to a variable,
  - function call,
  - return,
  - arithmetic and logical operations (e.g. addition, comparison, etc)

Note:

Copying a tuple / list / string takes time that is order of its length.

Standard tuple copying in Python is shallow, even with nested structure.

Example: Computing largest element in a list

```
def max_list(L):

   # assuming list is non-empty

   max = L[0]

   for i in range(1,len(L)):
      if(L[i] > max):
         max = L[i]

   return max
```

Number of primitive operations: At most c*len(L), for some constant c

How many primitive ops in this code?

```
def palindrome_recursion(s):

   if(len(s) <= 1):
      return True

   if(s[0] != s[-1]):
      return False

   t = s[1:len(s)-1]
   return palindrome_recursion(t)
```

How many primitive ops in this code?

```
def palindrome_recursion(s):

   if(len(s) <= 1):
      return True

   if(s[0] != s[-1]):
      return False

t = s[1:len(s)-1]
return palindrome_recursion(t)
```

Each recursive call creates a new substring via slicing (s[1:len(s)-1]).

This copies in total (n-2)+(n-4)+…. characters

How many primitive ops in this code?

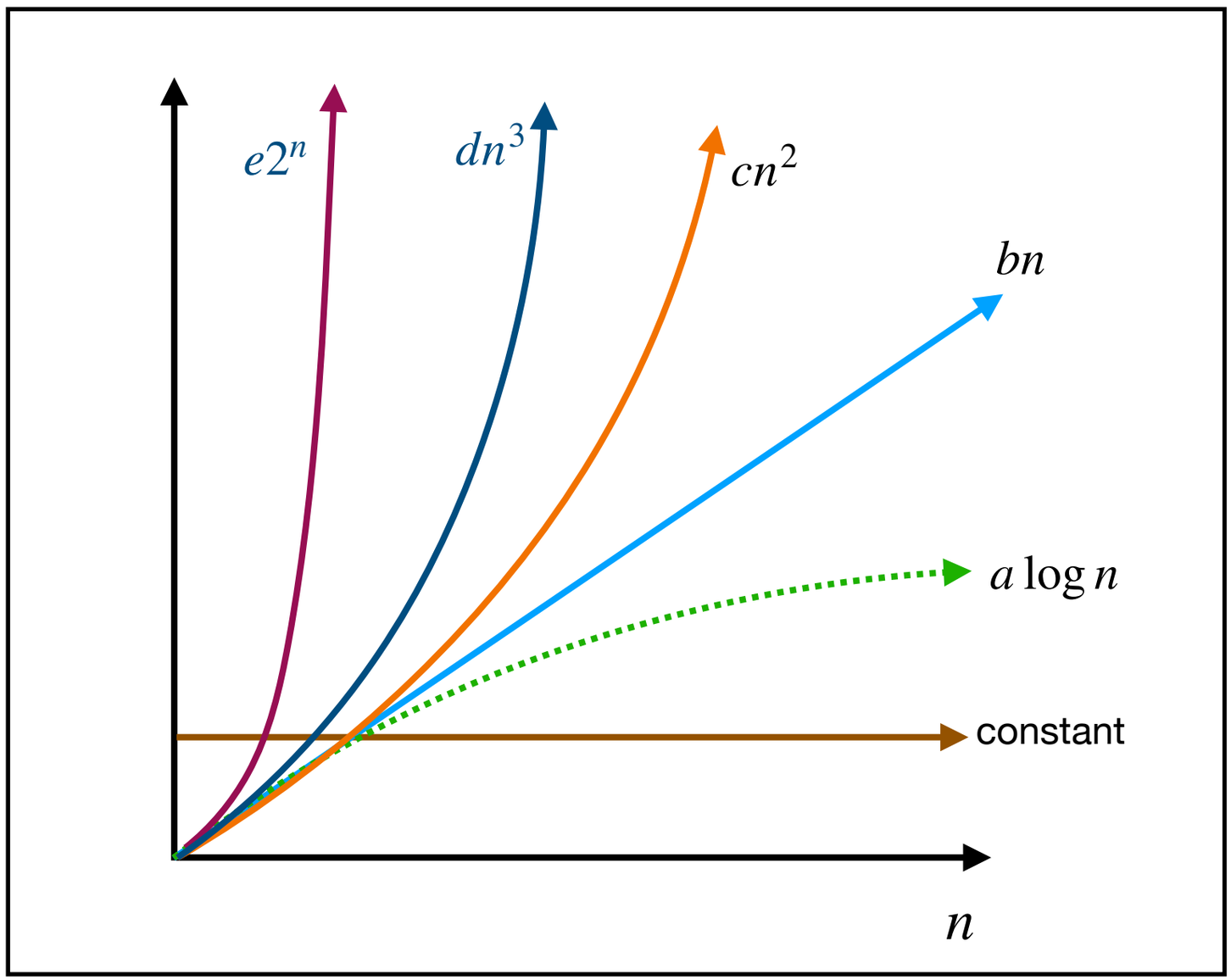
```
def palindrome_recursion(s):

   if(len(s) <= 1):
      return True

   if(s[0] != s[-1]):
      return False

   t = s[1:len(s)-1]
   return palindrome_recursion(t)
```

Number of primitive operations: At least $c*len(s)^2$, for some +ve constant c

Each recursive call creates a new substring via slicing (s[1:len(s)-1]).

This copies in total (n-2)+(n-4)+…. characters

Plot of different Functions



For constants a, b, c, d, e, eventually we have :

$$a \log n < bn < cn^2 < dn^3 < e2^n$$

How many primitive ops in this code?

A program that takes as input a positive integer n, and prints the smallest y such that $y^2 \geqslant n$

```
n = int(input("Enter number: "))
y = 0

while (y*y < n):
    y = y + 1

print("Smallest y such that y*y >= n is:", y)
```

Number of primitive operations: At most $c\sqrt{n}$, for some +ve constant c

Checking primality faster

```
n = int(input("enter number:"))

ans = True

for i in range(2,n):

   if (n%i == 0):
      ans = False
      break



if(ans):
   print(n,"is prime")
else:
   print(n,"is not prime")
```

At most c*n primitive operations,

for some +ve constant c

```
n = int(input("enter number:"))

ans = True

for i in range(2,n):

   if (n%i == 0):
      ans = False
      break

   if (i*i > n):
      break

if(ans):
   print(n,"is prime")
else:
   print(n,"is not prime")
```

At most $c\sqrt{n}$ primitive operations,

for some +ve constant c